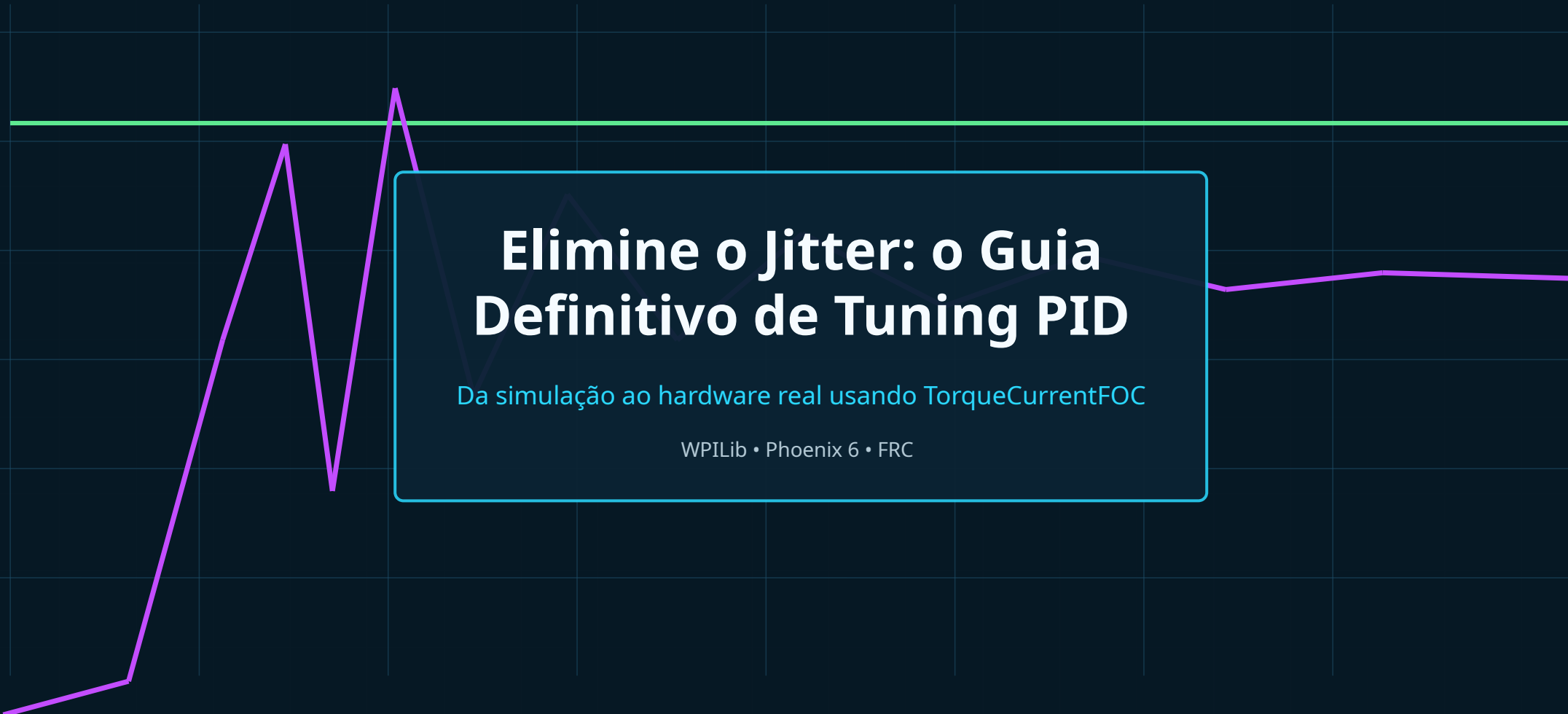




Elimine o Jitter: o Guia Definitivo de Tuning PID

Da simulação ao hardware real usando TorqueCurrentFOC

WPILib • Phoenix 6 • FRC

O Dilema da Pré-Temporada

SIMULAÇÃO

**Código pronto.
Mecanismo ainda em fabricação.**



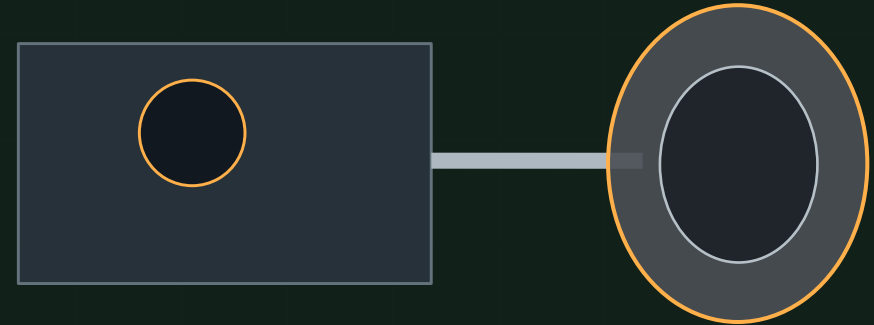
Inércia

Atrito

Carga

Ruído

HARDWARE REAL



A simulação não “substitui” o robô. Ela antecipa comportamento e gera ganhos iniciais mais seguros.

Objetivo: chegar ao hardware entendendo as forças do mecanismo — e não apenas testando números.

CTRE Phoenix 6

Mudança de Paradigma: Tensão vs. Corrente de Torque

A correção principal: corrente de torque → torque; a aceleração depende do torque resultante e da inércia.

CONTROLE DE TENSÃO

Saída em volts



Tensão → corrente do motor → torque
(com influência da Back-EMF)



A velocidade afeta a tensão necessária.
Por isso kV é importante no controle por tensão.

A aceleração é controlada INDIRETAMENTE.

TORQUE CURRENT FOC

Saída em amperes de torque



Torque current → torque do rotor



Torque resultante ÷ inércia → aceleração

Atrito, carga, gravidade e limites elétricos também
influenciam a aceleração real.

**TorqueCurrentFOC controla aceleração de forma
muito mais direta.**

Velocidade — Estratégia 1: Entender o efeito do kP isolado

Útil para aprender a relação erro → amperes. Não é o tuning final recomendado.

1 Defina um alvo

Ex.: 100 RPS. Comece com feedforwards em 0 para enxergar o efeito do feedback.

2 Aplique kP

Ex.: erro de 20 RPS e saída desejada de 10 A → $kP \approx 0,5 \text{ A/RPS}$.

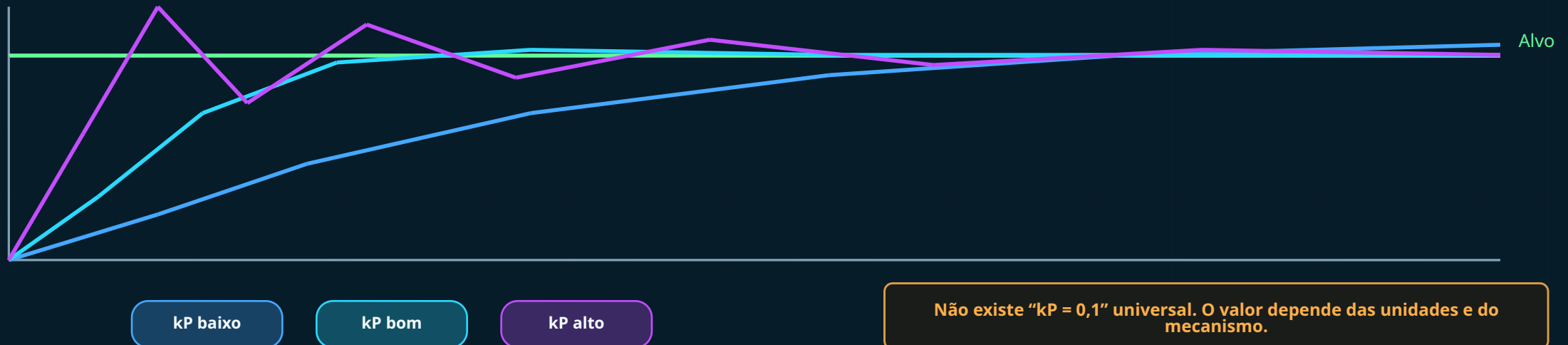
3 Observe resposta

kP maior corrige o erro mais rápido, mas pode aumentar oscilação e ruído.

4 Use como referência

Depois, ajuste kS/kV e retorne ao kP para a resposta final.

Exemplo visual: mesmo alvo, três valores de kP

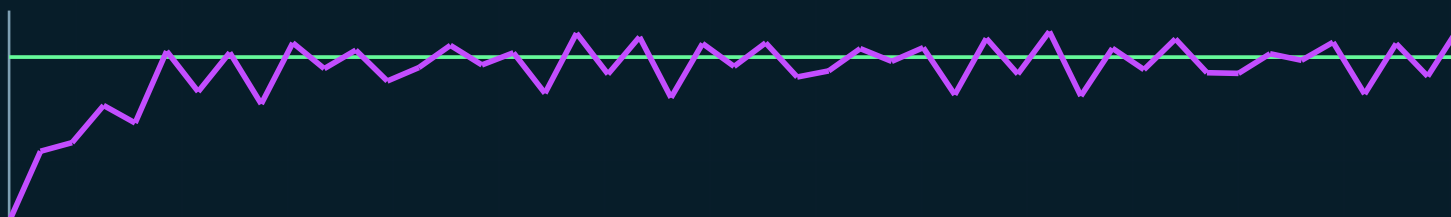


O Choque de Realidade: Ruído, Flexão e Vibração

SIMULAÇÃO IDEAL



HARDWARE REAL



NÃO FILTRE DEMAIS

- Filtro excessivo suaviza o gráfico, mas adiciona atraso.
- Folgas, flexão de eixo, corrente e sensores reais criam ruído.
- Primeiro: melhore rigidez, alinhamento e montagem.
- Depois: tune o loop para responder à realidade, não ao gráfico "bonito".

Ruído ≠ falha automática do PID

Velocidade — Estratégia 2: A Física do Feedforward

No TorqueCurrentFOC, k_S e k_V compensam forças físicas antes que o feedback precise corrigir.

k_S — atrito

Compensa a força necessária para iniciar/manter o movimento contra atrito.

MOTOR + MECANISMO



k_V — arrasto viscoso

Compensa forças resistivas que crescem com a velocidade.

$k_S = \text{output}$
No TorqueCurrentFOC: tipicamente em A.

$k_V = \text{output} / \text{velocidade}$
Ex.: A/RPS. Pode ser 0 em mecanismos com pouco arrasto.

Correção importante: “arrasto aumenta com a velocidade” — não assuma comportamento exponencial.

Decodificando kS, kV e kP

As unidades ajudam a conferir se o tuning faz sentido.

kS — Atrito

$$kS = \text{output}$$

TorqueCurrentFOC: A

Compensa a parcela quase constante de atrito. Em velocidade baixa, seu efeito domina.

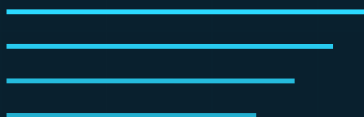


kV — Arrasto

$$kV = \text{output} / \text{velocidade}$$

Ex.: A/RPS

Compensa drag/atrito viscoso. Em TorqueCurrentFOC pode ser pequeno ou até 0.

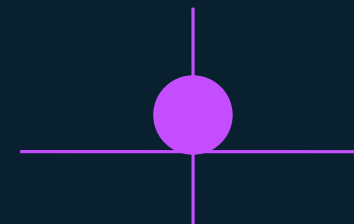


kP — Correção de erro

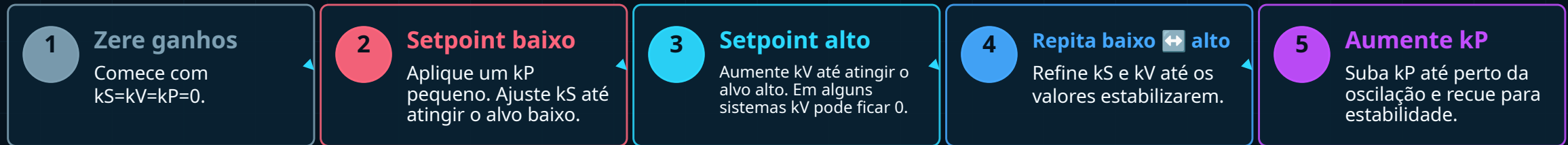
$$kP = \text{output} / \text{erro}$$

Velocidade: A/RPS

Converte erro de velocidade em corrente de torque corretiva.



Algoritmo de Tuning de Velocidade — TorqueCurrentFOC



POR QUE ESSA ORDEM FUNCIONA?

- k_P pequeno evita “mascarar” feedforwards ruins.
- Em baixa velocidade, k_S domina; em alta, k_V se torna mais visível.
- Depois de acertar as forças físicas, k_P passa a corrigir principalmente os erros restantes.
- Sempre valide no intervalo real de velocidades do mecanismo.

Regra prática

$k_S / k_V = \text{física do mecanismo}$
 $k_P = \text{rapidez para remover erro}$

Não fixe k_P inicial como “0,1”. Use um valor pequeno adequado às unidades configuradas.

Por que kD normalmente fica em 0 no exemplo de Velocidade?

Correção: “normalmente não necessário” é diferente de “inútil”.

FLYWHEEL / VELOCIDADE SIMPLES

No procedimento manual da CTRE, o foco final é:

$$kS + kV + kP$$

O mecanismo mantém velocidade quando o torque resultante vai para perto de zero. O kP corrige o erro de velocidade; kD não é parte principal desse fluxo.

Resultado: menos complexidade quando o sistema já está estável.

QUANDO NÃO GENERALIZAR

kD pode ser útil em outros sistemas ou critérios de desempenho.

- Não trate kD como “proibido”.
- Valide ruído, latência e dinâmica do mecanismo real.
- Em controle de posição por TorqueCurrentFOC, kD passa a ser essencial para amortecimento.

Mensagem correta: “normalmente 0 em velocity simples”.

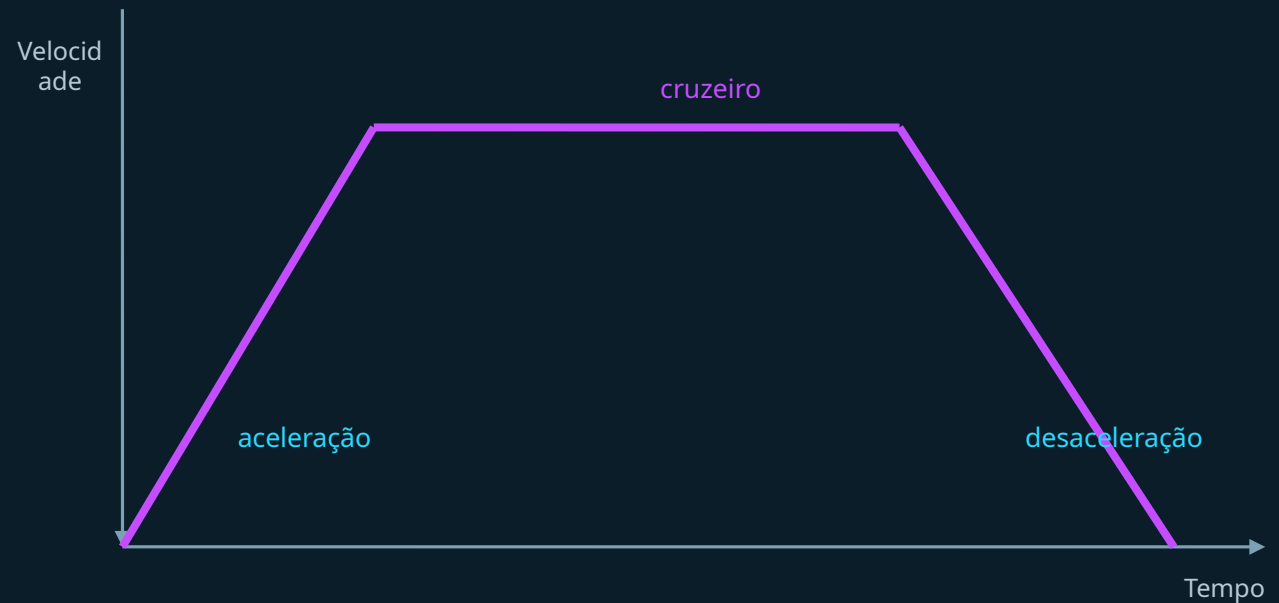
Próximo Nível: Controle de Posição com Motion Magic

QUANDO USAR

- Elevadores
- Braços
- Turrets
- Steer de Swerve
- Mecanismos que precisam chegar à posição sem “bater” no alvo

Motion Magic gera um perfil de movimento com velocidade e aceleração controladas.

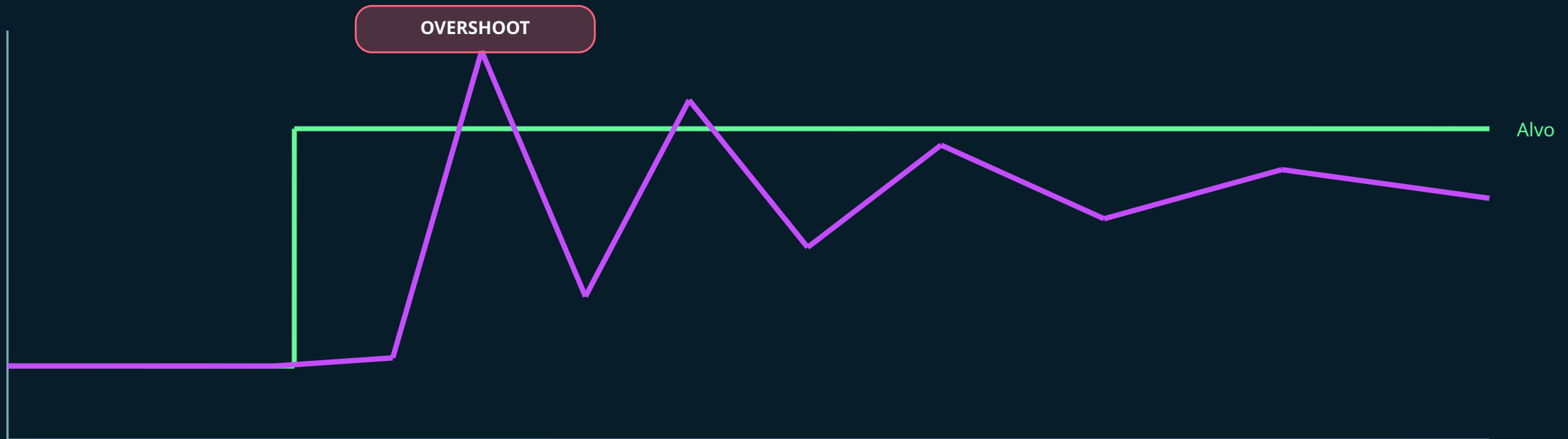
Perfil trapezoidal de velocidade



Perfil = setpoints previstos → oportunidade para kA e kV trabalharem como feedforward.

A Física da Posição no TorqueCurrentFOC

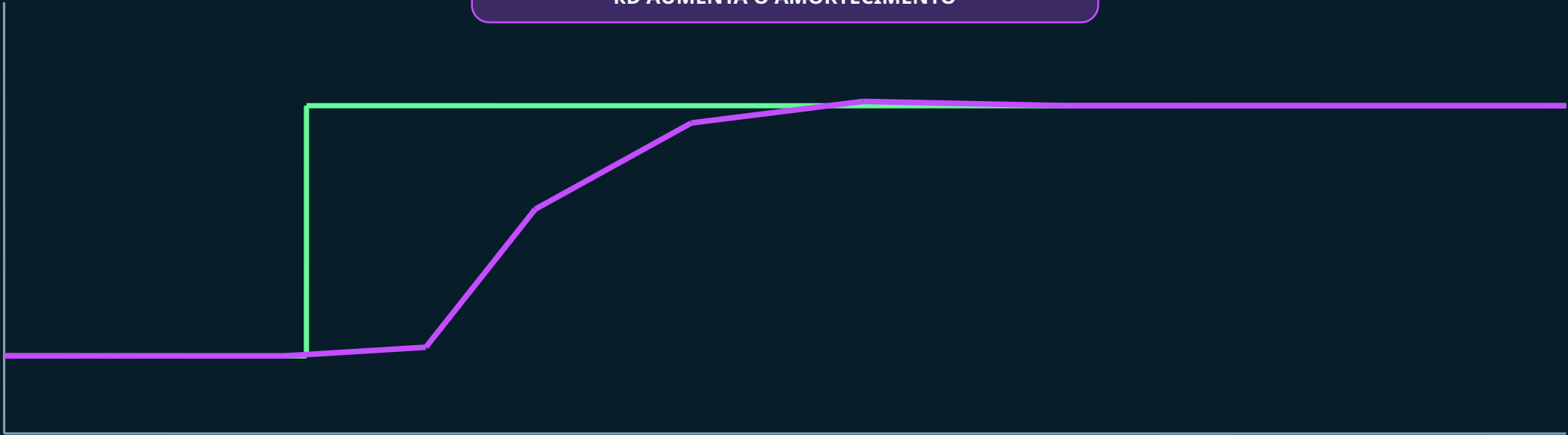
Sem o amortecimento natural da tensão, k_P sozinho tende a ultrapassar o alvo.



Por quê? No torque-based position control, o sistema perde grande parte do amortecimento natural associado à Back-EMF. Com k_P alto e sem k_D , o robô “passa” do alvo e oscila.

A Entrada do kD: o “Freio” do Controle de Posição

kD AUMENTA O AMORTECIMENTO

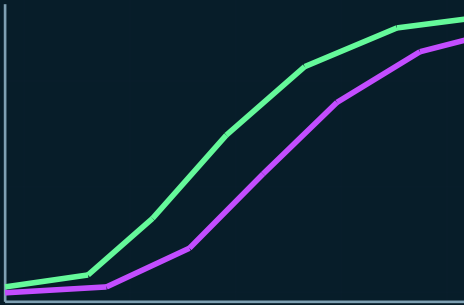


O kD age sobre a variação do erro — na prática, “segura” o mecanismo quando ele se aproxima rápido demais do alvo. A CTRE recomenda kD para TorqueCurrentFOC em loops de posição.

Dominando kA: Feedforward de Aceleração

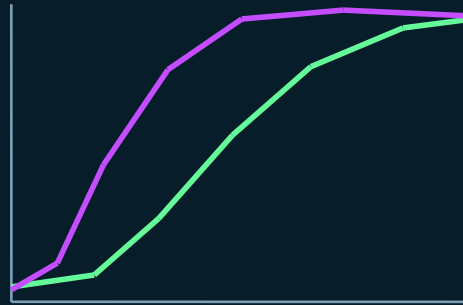
kA compensa a inércia e é a “espinha dorsal” do acompanhamento de um perfil.

kA BAIXO



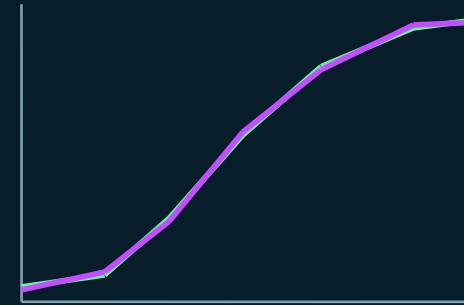
Resposta atrasada

kA ALTO



Resposta antecipa / exagera

kA CALIBRADO



Perfil acompanhado

$kA = \text{output} / \text{aceleração}$ → em TorqueCurrentFOC, a saída está em amperes. Ajuste kA até a inclinação da velocidade real acompanhar a inclinação do perfil.

Matriz Mestre de Tuning — TorqueCurrentFOC

Versão revisada: evita tratar kV/kD como regras absolutas.

GANHO	VELOCIDADE — TorqueCurrentFOC	POSIÇÃO / MOTION MAGIC — TorqueCurrentFOC
kS	✓ Normalmente usado Compensa atrito.	✓ Normalmente usado Compensa atrito.
kV	● Só se houver drag significativo Pode ser 0 (ex.: alguns swerves).	● Muitas vezes pode ser 0 Use se houver drag durante cruzeiro.
kA	● Geralmente 0 em velocidade constante Útil se houver aceleração/setpoint perfilado.	✓ Muito importante no perfil Compensa inércia/aceleração.
kP	✓ Feedback principal Converte erro de velocidade em torque.	✓ Feedback de posição Reduz erro em relação ao alvo.
kD	● Normalmente 0 no flywheel simples Valide conforme dinâmica real.	✓ Essencial para amortecimento Reduz overshoot/oscilação.

Regra maior: tuning é sobre a física e a resposta do mecanismo — não sobre marcar todos os ganhos como obrigatórios.

CTRE Phoenix 6

Diagnósticos Finais e Princípios de Laboratório

01

SIMULAÇÃO = PONTO DE PARTIDA

Use WPILib + Phoenix 6 para entender o mecanismo e chegar a ganhos iniciais seguros. Valide e refine no hardware real.

02

PENSE EM FORÇAS E UNIDADES

Em TorqueCurrentFOC, o output é corrente de torque. kS, kV, kA, kP e kD devem ter unidades coerentes com seus setpoints e erros.

03

FEEDFORWARD ANTES DO “BRUTE FORCE”

Compense atrito, arrasto e inércia primeiro. Depois use feedback para remover o erro residual.

04

RUÍDO É REAL

Não mascare problemas mecânicos com filtro pesado. Rigidez, alinhamento, sensores e latência fazem parte do controle.

Objetivo final: um mecanismo previsível, estável e repetível — primeiro na simulação, depois no robô.

CTRE Phoenix 6